1. Introduction

1.1 Introduction

Healthcare systems are an integral part of society, and the proper functioning of healthcare record management is essential for appropriate treatment. At the same time, many small clinics and hospitals often use paper registers and physical files to store and manage information about appointments, prescriptions, and patients' details. Such organization of record keeping requires much effort and is associated with numerous errors due to misplacement or incorrect writing. At the same time, the number of patients increases, and the staff needs more time to organize appointments. Herewith, the current project, Medical Appointment & Prescription system, is the application that was created in order to improve a hospital's workflow by introducing a desktop application that would allow storing and managing patient's information, doctor's appointments, and medical records. The system is designed on Java Swing, Core Java, JDBC, and MySQL to ensure the effective functioning of the organization.

The main purpose of Medical Appointment & Prescription system is to provide healthcare institutions with a reliable way to control and follow patients' treatment, register new ones, write and keep doctors' appointments, and store doctors' prescriptions electronically. First, the system allows patients to register and provide personal information, and doctors can enter their data to view their availability for appointments. In addition, the application helps medical workers manage the schedule, mark free hours, and record patients' visits to the hospital. Moreover, Medical Appointment & Prescription system helps set treatment courses, gives reminders about medication, and provides doctors with an option to see the treatment history of patients. Following the treatment course, the program helps send messages about necessary drugs and the time of the doctors' visits. At the same time, the staff can check patients' treatment and current medication at any time because this information is in the database and is available instantly. Herewith, the system has a particular option to print reports in PDF format for further inspection.

In a broader context, the current project aims at improving the general functioning of healthcare institutions by reducing paperwork, minimizing clerical errors, and ensuring a fast workflow for both patients and doctors. Besides, it also demonstrates how Java language and databases can be used in applications, in general, and in healthcare, specifically.

1.2 Objectives

The main purpose of the Medical Appointment & Prescription System is to design a simple and effective desktop application to help clinics with their day-to-day activities and reduce paperwork.

The objectives of this project are:

- Designing a computerized database that would hold patient and medical records, with access only given to authorized personnel.
- Maintaining a database of doctors' information, schedules, and availability.
- Developing a system that would make it easier to schedule appointments, avoiding double-booking and long waits.
- Creating a system that would store digital copies of prescriptions and patient history, remind patients of medication, and track status of appointments, with reports generated in a PDF format for easy printing and storing of medical records.
- Making the application easy to use and intuitive so that it reduces human error and increases overall productivity.
- To develop a system that would allow for a faster search of patient information rather than having to go through paperwork.
- To develop a self-updating calendar for the receptionist to know which time slots are available for scheduling appointments, as well as to avoid double-booking doctors.
- To develop means of restricting access to patient records so that only authorized personnel can view or update medical records.

1.3 Scope of the Project

The scope of the Medical Appointment & Prescription Information System determines its features and directions of development. It is a desktop application designed for small-to-medium-sized hospitals, clinics, and private practices.

i. The Features within the Scope (Will Be Done)

- **User Login Page:** A login page, where users can enter their credentials to gain access to the application. Authentication mechanism (based on admin/doctors) to ensure confidentiality of medical information.
- **Patient Operations**: A set of operations to register/register new patients or keep their personal information up to date.Maintaining patients' electronic medical records.
- **Doctor Operations:** Set of operations to keep track of doctors' schedules, manage the up-coming appointments, and update personal information, if needed.
- **Appointment Operations:** A set of operations to book appointments for patients and maintain their status (booked/cancelled).
- **Prescription Operations:** A set to create electronic prescriptions, keep track of the past ones, and set reminders for patients to take their medication.
- **Reports :** An option to generate reports in PDF format.
- **Database and Front-end:** Database (MySQL) with JDBC and Front-end (Java Swing and Core Java).

ii. The Features outside the Scope (Will Not Be Done)

- The ability for patients to log on to the system from their phones/tablets/laptops by way of an app or a web version: The application is strictly a desktop application for use by staff in the hospital or a clinic. It is not designed to be used by patients to schedule their own visits.
- **Ability to pay for consultation/appointments online:** The scope of this project does not include processing payments or insurances, either via online services or in-clinic.
- **Managing inventory within the pharmacy (medicine stock list) or billing (generating invoices):** The system is intended to manage only the prescriptions, not to keep track of the stock of medication available in the hospital pharmacy or generate any billing-related documents for patients or departments.
- Hosting the application on a cloud server (AWS, etc) instead of a local MySQL server (desktop): The system is hosted locally, on a desktop computer with a MySQL server, rather than a remote server.

1.4 Existing System and Limitations

i. The Existing Manual System

Most of the small clinics and hospitals currently use a manual system of managing patient records in file or register and also managing appointments manually using a hand written diary. The doctors also have to manually record prescriptions they give on paper pad.

ii. Limitations of the Existing System

- Time consuming since they require searching through files for information
- Manual work leads to errors and loss of information in files
- Appointment system is not automated leading to appointment clashes and also making people waste their time waiting for appointments.
- Prescriptions are handwritten making it easy to miss them or misread them.

- There is no reminder system in place for taking drugs and attending to appointments.
- Easy access to patient records which are in paper form and also easy to damage.

1.5 Proposed System and Advantages

i. Proposed System:

To address the limitations of manual processing, the research work has proposed a computerized desktop application called Medical Appointment & Prescription System. It has been developed using Java Swing, Core Java, JDBC, and MySQL. It aims to offer a secure, centralized digital platform for managing patient registration, doctor availability, appointment management, prescription, medical history, and generating reports in PDF format.

ii. Advantages Of The Proposed System

- **Fast record retrieval:** Through online search, hospital staff can access patient's medical data instantly instead of searching for physical files.
- **Improved data accuracy:** Storing patients' medical data in a database reduces the possibility of mistakes, eliminates the need for repetitive records, and offers confidentiality.
- **Avoids appointment clashes:** This system can offer automated appointment confirmation and scheduling. In addition, it eliminates the chances of repeating schedules and reduces waiting time.
- **Easy record of prescriptions:** It allows doctors to keep track of medicine easily by generating digital prescriptions and reviewing medical history before deciding or prescribing medication.
- **Reminders of medicine:** The system features an alert system that reminds patients of the dosage schedules and tracks their treatment progress in terms of time.
- **Generation of reports:** This system is designed to offer a fast way of generating reports in PDF format, making it easy to store or share medical records.
- **Enhanced security:** The proposed system will be offering restricted access to the database, ensuring confidentiality of health records as opposed to the manual system in which patients' medical data can be accessible to everyone.

1.6 Feasibility Study

Prior to developing the Medical Appointment & Prescription System, a feasibility study was carried out to establish whether the proposed project is simple, cost-effective, and easily achievable within the set time frame. The following aspects of the study were considered:

i. Technical Feasibility

- This project will be developed using the following technologies:
- Front end – Java Swing
- Back end – Core java
- Database – MySQL
- JDBC – Java Database Connectivity
- The tools are open-source and their use is well-documented. Therefore, there is no technical challenge in adopting them.
- Conclusion: The project is technically feasible.

ii. Economic Feasibility

The following aspects of this project are economically feasible:

- The tools and technologies used in the development of this project are open-source. Therefore, there are no costs associated with their acquisition. Although the project will require a computer to host the application, most members of the staff already possess this hardware.
- Conclusion: This project is economically feasible.

iii. Operational Feasibility

The following aspects of this project are operatively feasible:

- The application will be developed using Java Swing, a tool that is easy to use and well-documented. It will be operated by hospital receptionists, administrators, and doctors. It will replace the manual register currently used in most clinics and hospitals. Therefore, it will not be challenging for these professionals to operate the system.
- Conclusion: The project is operationally feasible.

iv. Schedule Feasibility

The following aspects of this project are feasible in terms of schedule:

- The project will be completed within the set time frame (August to December). The time allocated for the project is sufficient for the two students working on it. All aspects of the project – requirements, design, coding, integration, and testing – will be completed before the submission date.
- Conclusion: The project is schedule-feasible.

2. Functional Requirements

The functional requirements specify the functions and actions that the Medical Appointment and Prescription System should perform to satisfy the end users, including hospital staff, doctors, and patients.

2.1 User Authentication & Authorization

The system should have a login portal where users can access the application.

- It should authenticate valid users from the database who can enter through the login portal by using their credentials.
- The system should also authorize users depending on the roles assigned to them and deny access to unauthorized users to the admin and doctor panel.
- It should simultaneously deny access to the main dashboard of the application when the login details do not match the database.
- The system should also allow the users to log out of the application at any time they want.
- It should allow the admin to add, update, and delete the staff and receptionist's login records.

2.2 Patient Registration

- The system should provide a registration portal where new patients can submit their details to the database
- It should collect patient information including patient names, age, gender, contacts, and addresses
- The system should generate a unique patient ID upon the successful submission of their details to the database;
- It should also ensure that users fill the mandatory fields with their contact information or any other personal details;
- Finally, the system should store the personal information of new patients in the MySQL database using JDBC.

2.3 Patient Records

- The system should provide a records folder to store the medical records of patients
- It should allow hospital staff to search for patient records using their unique patient ID or names;
- The system should also allow hospital staff to update patient contact information or any other relevant personal details;
- Finally, it should allow the staff to view patient records on screen.
- In addition, the system should ensure that all the patient's previous medical records are up-to-date throughout the system.

2.4 Doctor Management

- The system should allow the admin to add a doctor's profile to the application database.
- It should collect the doctor's profile details, such as doctor's name, specialization, personal information, and doctor's working department.
- The system should update the profile of the doctor if there are any changes in the contact information or working department of the doctor.
- It should also provide a list of all the available doctors in the hospital or clinic in an easily accessible manner. The system should also allow the admin to delete the doctor's profile if they change their places of work.

2.5 Doctor Availability

- The system should allow doctors to set their working schedules for easier access to the hospital staff
- It should also allow doctors to configure whether they are available or not on specific dates and days;
- The system should update the availability of doctors to ensure seamless scheduling procedures; it should also stop staff from scheduling appointments on days when the doctor is not available;
- Finally, the system should store doctor's availability in the database for future reference.

2.6 Appointment Scheduling

- The system should give the hospital staff an option to schedule appointments for patients who need medical services.
- It should also allow the staff to choose from the available time slots according to the doctor's schedules;
- The system should prevent the staff from scheduling two patients at the same time with one doctor;
- It should also minimize the waiting time of patients by organizing the time schedules accordingly;
- Finally, the system should store the appointment records with the patient ID and doctor ID in the database.

2.7 Prescription

- The system should give doctors the option to provide electronic prescriptions for patients who need medication.
- It should also give doctors the option to enter the medicines, directions, and other relevant information for patients' treatment;
- The system should link the prescriptions to patient ID and the day or date of the appointment;
- It should also allow doctors to review and finalize the electronic prescription for each patient;
- Finally, the system should store the electronic prescriptions for future reference securely.

2.8 Prescription History

- The system should store all the past prescription history of every patient in the medical records;
- It should allow doctors to view the prescription history of patients before giving new medication; the system should organize the prescription history depending on the year it was issued;
- It should also ensure that the medical history of patients is available upon request;
- Finally, the system should provide an easy option for doctors to locate the past or current medical records of patients.

2.9 Medicine Reminder

- The system should allow staff and doctors to remind patients of their medicine dosage.
- It should provide medicine reminders by fetching the patient's electronic prescriptions data.
- The medication reminder should alert the staff and doctors in the application screen to remind the patient of their medicine.
- It should also assist hospital staff and doctors to monitor and remind the patient of the dosage of the medication and pass the information to the patient.
- Finally, the system should record the status of the medicine reminder to ensure that each patient takes their medicine at the exact time.

2.10 Medical History

- The system should store the medical history of every patient in the medical records.

- It should also update the medical history of patients according to the doctors' needs and the treatment undergone by the patient;
- The system should give doctors the option to view the medical history of the patients before initiating any medication;
- It should ensure that the medical history of the patient is attached to his or her personal information;
- Finally, the system should allow the doctors to add new medical history notes to every patient after every examination.

2.11 Appointment Status

- The system should provide an option to track the status of every appointment, including cancelled, pending, finished, or confirmed.
- It should allow staff to manually update the status of every patient's appointment as they schedule or complete each day's appointments.
- The system should display the status of each appointment using various colors to indicate the meaning of each status.
- The system should automatically archive completed or cancelled appointments from the dashboard view after 24 hours while keeping the data available for medical history and report generation purposes.
- Finally, the system should allow the staff and doctors to monitor every day's activities and the status of every patient's appointment.

2.12 PDF Report

- The system should provide an option to generate a PDF report for every patient's appointment and prescription.
- It should also allow the generation of a PDF file containing the patient's personal information, doctor's personal information, patient's examination, and the medication the patient needs.
- The option should make it easy for users to print or save the reports for future reference anytime they need. The system should also store the reports in the system to help in easy retrieval when needed.

3. Non-Functional Requirements

The non-functional requirements specify the quality attributes, constraints, and characteristics of the Medical Appointment & Prescription System.

3.1Performance Requirements

- The application must respond to patient-record queries and database searches within less than two seconds.
- The application must allow multiple users to operate simultaneously without slowing down the processes on a busy server.
- The software should load and open the graphical user interface quickly on a typical personal computer.
- The program must update and record appointment data in real-time to avoid processing delays.

3.2 Security Requirements

- The software must require users to enter a valid user name and password to eliminate unlawful access.
- The application must use role-based access control to authorize only verified doctors and nurses to update or view patient data.
- The Java database connectivity must protect the software's credentials to prevent illegal access to internal tables.
- The system must prevent unauthorized alteration of records by ensuring steady authentication of authorized users.

3.3 Usability Requirements

- The application must use standard java swing controls and provide intuitive navigation between features, forms, and menus.
- The software must provide informative messages and error alerts to guide the users to correct input errors.
- The application must be designed to minimize workload for hospital receptionists and doctors while providing relevant data and maintaining privacy.
- The program must use simple and standard layouts, typographic formats, and table designs to reduce the mental effort of interacting with the software.

3.4 Reliability and Availability Requirements

- The application must be reliable by saving and storing patient data accurately without corrupting files during power outages and abnormal shutdowns.
- The medical application must save all patient records, schedules, and prescriptions in the MySql database accurately without deletion.
- The local desktop application version must be available for internal hospital use at all times.
- The software must handle all runtime errors gracefully by recording all error messages rather than terminating abnormal functions.

3.5 Maintainability and Scalability Requirements

- The application must be written using modular programming codes to allow easy maintenance and scalability.
- The software must use standard database management systems that can easily accommodate more patient data records in the future.

- The program’s source codes must be well-commented and use standard identifiers to allow easy modification.
- The application must allow separate modules to update separately while maintaining stable system functionality.

4. System Architecture

System architecture describes how the Medical Appointment & Prescription System front end, back end and database is interconnected to manage the flow of information.

4.1 The overall system architecture

- The Medical Appointment & Prescription System operates on a 3-tier desktop architecture model to segregate presentation, rules, and data storage functionalities.
- The system is designed to operate on a local computer platform where front-end views, Java core and database tables exist as separate entities.
- This ensures that changes made to the front-end visuals or Java swing interface do not affect the lower layers of the application.

4.2 Presentation layer and UI

- This layer is constructed using java swing to accommodate the application's graphical user interface.
- The front end is the working platform for clinic receptionist, administrators and doctors to perform their routines such as capturing new patient information, scheduling doctor appointments and capturing prescription details.
- It captures all the user inputs and interactions before passing the information to the application core.

4.3 Business logic layer and the core

- Core Java is used to develop the application's business logic layer.
- This layer accommodates all the algorithms, validations, and processes required to support the functionalities provided by the medical appointment system.
- It receives user requests from the presentation layer, carries out necessary validations and checks including verifying user access rights and detecting appointment scheduling conflicts.
- The business logic layer also manages automated processes including converting patient data into a PDF file format and scheduling reminder alarms for patients' medication.

4.4 Data access layer and database

- MySQL is used as the database management system to store all patient, doctors' and pharmacy records permanently.
- The data access layer connects the MySQL database to the core java application through JDBC (Java Database Connectivity) drivers.
- This layer supports the creation of databases and tables for the medical appointment system.
- It also facilitates the modification of existing database systems including managing patient records, doctor's records, appointment information, and managed prescriptions in a relational database management system.

4.5 Data flow around the application

i. Input

- A user performs an action such as saving a new patient record in the medical appointment system.

ii. Process

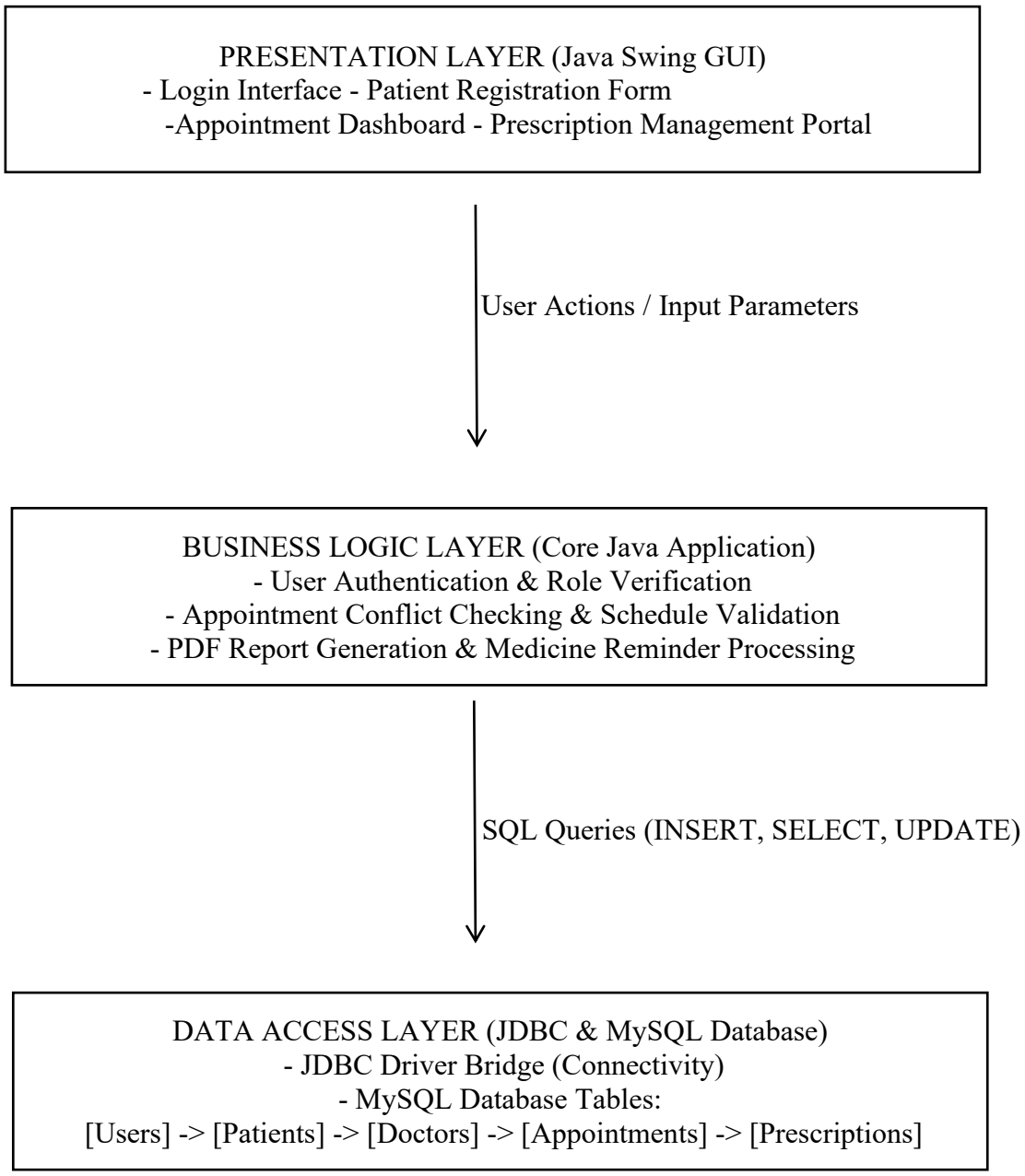
- Java swing object receives the input as a parameter and passes it to the application core.

- The core uses Java code to carry out input validations, checking and/or eliminating any conflicts that may arise when carrying out the operation.
- The application core prepares SQL statements that will be used to execute the database functions.

Output

- The database receives the SQL instruction, executes it and finally returns the generated results to the application core through JDBC.
- The medical appointment system then updates the database tables and eventually outputs the final results to the user through Java Swing interface.

4.6 Architecture Diagram



5. Database Management

The database management section below describes how the Medical Appointment & Prescription System will store, organize, and protect its data. The software uses MySQL as its database management system which uses relational tables to store information securely and quickly, while also being compatible with Java through JDBC.

5.1 Overview of Database Management

- MySQL, a reliable open-sourced Relational Database Management System (RDBMS), will be used to store all data on a local server,
- The database uses a normalized schema to avoid repetition of data, thus ensuring users edit data in one place and the changes are reflected in all sections requiring it. For instance, a doctor's schedule and a patient's address will be stored once and used to update related files when needed
- Communication between MySQL and the Core Java application will be done using JDBC (Java Database Connectivity) which executes SQL queries to perform the following operations: Create, Read, Update, and Delete (CRUD).

5.2 Entity-Relationship (ER) and Data Mapping

- The database was designed using entities that will be found in the hospital environment including Users, Patients, Doctors, Appointments, and Prescriptions.
- One-to-Many relationships were designed to indicate how one entity can relate to many others. For instance, one patient can have several appointments, but a doctor can attend to many patients in a day.
- The database has implemented foreign keys to ensure referential integrity of the data. This can be seen in how patient_id and doctor_id are used to establish a link between appointments and the patients/doctors database instead of repeating the patient's name or the doctor's details.

5.3 Data Dictionary and Table Structures

- For the purpose of structured data storage, the following tables will be used by the system with the indicated properties:
- **Table 1: Users (Authentication Data)**

Column Name	Data type	Description
user_id	Primary Key, INT	A unique, auto-incremented ID identifying all staff members and doctors who use the software system.
username	VARCHAR, UNIQUE	A login name unique to the employee which they use to access the system
password	VARCHAR	A password to which only the employee knows which is used to gain access to the system
role	VARCHAR	This defines the level of access the employee has in the system, such as the examples: Admin, Receptionist, Doctor.

- **Table 2: Patients (Patient Demographics & Background)**

Column Name	Data type	Description
patient_id	Primary Key, INT	A unique ID issued automatically upon registration.
registered_by	FOREIGN KEY (references users.user_id), NOT NULL, ON DELETE RESTRICT, INT	Follows the staff or admin user account that initially created the patient entry.

full_name	VARCHAR	The full legal name of the patient.
age	INT	Basic demographic information for medical identification.
gender	VARCHAR	
contact_number	VARCHAR	A phone number to which the organization will send reminders about scheduled blood donations and medication doses.
residential_address	TEXT	The home address for official records.
blood_group	VARCHAR	Vital signs required in case of an emergency.
registration_date	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Record the exact date and time of record creation automatically

- **Table 3: Doctors (Professional Profiles & Schedules)**

Column Name	Data type	Description
doctor_id	Primary Key, INT	A unique identification number for medical staff.
user_id	Foreign key, INT	Establishes link between doctor's professional account and their personal account in the Users table
doctor_name	VARCHAR	The Doctor's full professional name.
specialization	VARCHAR	The medical department or specialization (e.g., Cardiologist, General Physician).
contact_number	VARCHAR	Internal communication information.
consultation_fee	DECIMAL	Standard charges for reporting purposes.
working_hours	VARCHAR	Specifies the operating hours of the doctor.
avaliability_status	BOOLEAN	Indicates whether the doctor is available for consultation.

- **Table 4: Appointments (Scheduling Data)**

Column Name	Data type	Description
appointment_id	Primary Key, INT	Unique tracker for every booked consultation.
patient_id	Foreign Key, INT	Link to the patient who booked the appointment
doctor_id	Foreign Key, INT	Link to the doctor to whom the appointment is assigned
appointment_date	DATE	The calendar date of the scheduled visit
appointment_time	TIME	The allocated slot to the visit to avoid overlaps
status	VARCHAR	The current status of the booking ("Pending", "Completed", "cancelled", etc.)
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Automatically stores the date and time at which the record was created

- **Table 5: Prescriptions (Medical Treatments & Records)**

Column Name	Data type	Description
prescription_id	Primary Key, INT	Unique key of the digital medical slip.
Appointment_id	Foreign key INT	Foreign key link to the associated appointment record.
patient_id	Foreign Key, INT	Indicates which patient was treated.
doctor_id	Foreign Key, INT	Specifies who prescribed them.
issue_date	DATE	Shows the date of prescription preparation.
Medicines_and_dosage	TEXT	Clear instruction on the name of drugs to be taken, the intervals between doses, and the duration of treatment.

clinical_notes	TEXT	Other symptoms, diagnoses, or instructions from the doctor.

- **Table 6 Medical Histories (Clinical Records & Patient History)**

Column Name	Data type	Description
history_id	Primary Key, INT	A unique, auto-incremented identifier for each clinical history record
patient_id	Foreign key INT	Establishes a relationship between the history record and the specific patient profile
doctor_id	Foreign Key, INT	Identifies the medical practitioner who recorded the history
recorded_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Automatically timestamps when the clinical notes were saved
Symptons	TEXT	Detailed record of the patient's reported symptoms
Diagnosis	TEXT	Clinical diagnosis determined by the attending physician
treatment_notes	TEXT	Additional treatment instructions, observations, or follow-up notes

- **Table 7 Medicines Reminders (Automate Treatment Tracking & alerts)**

Column Name	Data type	Description
reminder_id	Primary Key, INT	A unique key for every dosage reminder alert.
patient_id	Foreign key INT	Links the dosage reminder warning to the targeted patient.
prescription_id	Foreign Key, INT	Connects the reminder message to the prescription slip from which it was generated.
medicine_name	VARCHAR	The name of the drug or medication that is prescribed.
Dosage	VARCHAR	The dosage instructions and frequency of intake
scheduled_at	DATETIME	The date and time of the medication reminder.
snoozed_until	DATETIME	Manages temporary postponement or snooze status of the alert
Status	VARCHAR	Tracks the current state of the reminder (e.g., “Pending”, “Taken”, “Snoozed”).
Interval_hours	INT	Represents the time recurrence interval in hours between the scheduled dose.

Table 8 Medicine Dose Histories

Column Name	Data type	Description
history_id	Primary Key, INT	It is the unique identifier for a particular dose history log entry.
patient_id	Foreign key INT	It specifies the target patient ID that the particular dose record

		belongs to.
reminder_id	Foreign Key, INT	It indicates the related medicine reminder's unique identifier used to track the dose.
medicine_name	VARCHAR	This will contain the medicine's name that was administered or tracked.
Dosage	VARCHAR	It will store the sample's dosage instructions.
scheduled_at	DATETIME	It indicates the time scheduled for the dose.
given_at	DATETIME	It specifies the time when the dose was marked as given or skipped.
Status	VARCHAR	This indicates whether the scheduled dose was "Given" or "Skipped".

5.4 Data Integrity and Constraints

- **Primary Key Constraints:** Each table is designed with a unique Primary Key that employs an auto-increment feature to assure that there are no two identical entries in the database (for example, two patients with the same name).
- **Referential Integrity:** Foreign keys are used in the database to ensure that no orphan records can be created. For example, a patient cannot be deleted from the database if there are any records of his or her upcoming appointments.
- **NOT NULL Constraints:** All the fields that hold patient or user data are set to NOT NULL in the database to prevent incorrect data from being submitted through the Java web application.

5.5. Database Security and Optimization

- **Prevention of SQL Injections:** Since the Core Java application only uses JDBC PreparedStatement objects for all of its database operations, any user input (e.g., a search query or a username/password during the login) is sanitized before being sent to the MySQL database. This prevents SQL Injection attacks that could otherwise compromise the integrity of the database.
- **Database Indexes:** The database is designed in such a way that all frequently queried fields (e.g., patient_id in the Patients table or appointment_date in the Appointments table) are marked as indexes in MySQL. This allows for quick data retrieval even when the records in the hospital database are in the thousands or more.

6. User Documentation and Operational Guidelines

In this segment, the user documentation section will provide end-users with the information on how to operate the Medical Appointment & Prescription System in the most optimal manner. The described end-users include clinic receptionists, administrative employees, and medical practitioners.

6.1 Information on the System Deployment and Accessibility

- Medical Appointment & Prescription System is a desktop application installed on the local computer of the clinic or hospital
- It requires Runtime Environment (JRE) and local MySQL database server connected via JDBC drivers
- The software is launched either from an executable file or from an IDE (Eclipse or NetBeans) as a Java Swing application

6.2 Information on Logging in to the Software

- When launching the application, the user is prompted to login to the Medical Appointment & Prescription System. This login window does not allow outsiders to access the system's core functions
- The user has to enter their specific Username and Password combination in the fields provided
- After clicking the Login button, the access is granted to either doctor section, receptionist dashboard, or user management panel via Core Java method executed by clicking Login button
- The access is denied in case of an incorrect Password or Username with a pop-up warning message appeared "Incorrect Username or Password". The incorrect password field is cleared and the user has to enter the right credentials again

6.3 Information on Searching, Booking, and Updating the Patient's Profile

- By pressing the "New Patient" button in the main menu, the receptionist is capable of opening a particular form and entering all required personal details of the patient such as name, age, gender, contact information, address, and blood group
- By pressing the "Save" button, the Swing form executes a command to save this information after a brief validation of mandatory address and contact fields
- The receptionist is also capable of searching for patients by entering their names or patient IDs and updating their phone numbers or addresses in the search bar and pressing the "Update Profile" button

6.4 Information on Doctor's Profile and Schedule

- In order to create a doctor's profile, the administrator is capable of pressing the "Doctor Management" button in the menu bar and entering the doctor's specialization, name, and consultation fee
- The doctor profile is also capable of updating the working hours and days of a particular medical professional
- Once the doctor updates their personal schedule, the changes are immediately reflected in the database. In such a manner, the doctors' availability is dynamically updated and receptionists are not able to book the already occupied hours of medical practitioners

6.5 Information on Booking the Appointment

- The receptionist has to choose the patient's ID, doctor, desired day of the appointment from the calendar, and the available time slot from the dropdown list
- By pressing the "Book" button, the Core Java method checks the availability of the selected time slot in the MySQL Appointments database
- By pressing the "Book" button again, the receptionist is able to view booked slots in the dashboard with their current status ("Pending", "Confirmed", "Completed", or "Cancelled") depending on whether the

patient arrived at the clinic for the scheduled appointment. In such a manner, the receptionist updates the status of each appointment in the dashboard

6.6 Information on Generating a Prescription and Its History

- The doctor is capable of generating a prescription for the patient either during his arrival at the clinic or after the doctor's examination. In such a manner, the doctor has to press the prescription button in the dashboard section, which corresponds to the current patients
- By entering the drug name, dose, intake frequency, period of therapy, and additional notes, the medical professional has to click the "Save" button to store this information in the MySQL Prescriptions database
- The doctor is capable of viewing the patient's medical history to prescribe the required drugs
- By pressing the "Export PDF" button, the user will be able to view or save this particular prescription or medical history as a PDF file.

7. Testing and Quality Assurance

The testing and quality assurance will be done to ensure that the Medical Appointment & Prescription System works properly, taking users' inputs accurately and safely, and satisfying all the requirements ahead of its launch.

7.1 Testing overview

- The testing will be done to assure the proper functioning of all the parts of the software, including the user registration, patient identification, scheduling of appointments, and generation of PDF reports.
- The black-box testing will be applied to the functions of the Medical Appointment & Prescription System, and the white-box testing will be done to the system's inner functions.
- The main goal of the testing is to reveal all possible logical errors, syntactical mistakes, data incompatibility issues, and interface-related incidents to ensure the software's reliability and effective daily use**.**

7.2 Unit testing

- The logic within Java classes will be tested to ensure the accuracy of calculations and other operations.
- The connection to the database will be tested using JDBC drivers. The input fields of the forms will be tested to ensure the validation of the entered information.

7.3 Integration testing

- The integrations between the software's parts will be tested to ensure the proper flow of information from the Java Swing interface through the Core Java layer and JDBC bridge to the MySQL database.
- The interaction between the elements of the medical application will be tested to confirm their proper interrelation.
- For instance, the confirmation of the integrated actions, such as the indication of the availability of the chosen doctor after scheduling the appointment, will be taken to ensure the correct updating of the foreign keys of the related database tables (patient_id and doctor_id).
- Testing the proper exception handling mechanism of the application to ensure the absence of any abrupt system failures during the sudden appearance of database-related errors will be done.

7.4 System and function testing

- The functioning of the whole system will be tested under realistic conditions to check if all the operations, such as patient registration, doctor identification, scheduling of appointments, prescription of medication, and printing of reports, work correctly and consistently.
- The system's authorisation interface will be tested to ensure that the receptionist's user account can only access the functions available to them and not open other restricted parts of the application.
- The extreme values and symbols allowed to be entered into the system will be tested to confirm the absence of encoding and range-related issues.

7.5 Performance, usability, and stability testing

- The time response of the database will be tested to ensure that all the search operations will yield results within 2 seconds of the user entering their queries.
- The interface of the system will be tested to guarantee the convenience of the navigation and operation with the software in accordance with its intended use.
- The stability of the desktop application will be tested to confirm that it will not crash during heavy use.

7.6 Regression and acceptance testing

- The regression testing will be done to ensure that the recent modifications to the software do not violate the existing functions and interfere with the database operations.
- The user acceptance testing will be done to confirm that the developed application satisfies all the requirements and is ready to be launched.

8. Acceptance Criteria

The acceptance criteria section below contains specific requirements that the Medical Appointment & Prescription System must meet to demonstrate that it is fully finished and ready for end-users, stakeholders, and hospital administrators.

8.1 General Introduction to Acceptance Criteria

- Acceptance criteria are standard requirements that any software must meet to satisfy a buyer's needs and prove that it is fit for purpose and use
- This is a checklist that stakeholders will use to test the final system's features and make sure that all requirements have been implemented before accepting delivery or signing the final payment voucher.
- Every requirement mentioned in this section must be tested and passed with no significant errors to signify successful completion and readiness for deployment in the target organization.

8.2 User Login Acceptance Criteria

- **AC-01**: As a Receptionist, I want to enter my login details and password that are encrypted in the MySQL database to ensure confidentiality
- **AC-02:** If a legitimate user logs in successfully, they should be redirected to the admin, receptionist, or doctor panel depending on their working section in the healthcare institution
- **AC-03:** When entering login details, the application should use a JDBC prepared statement to prevent SQL injection attacks hence protecting the database from being manipulated or hacked

8.3 Register New Patients Acceptance Criteria

- **AC-04:** When filling out the registration form for new patients, users should not be able to leave any fields blank or enter the wrong data type in any input field
- **AC-05:** When clicking the save button, the application should automatically generate a unique ID to every patient and store it in the database
- **AC-06:** Users should search for patients using their IDs or names and update their contact information or fill out their background medical history in the database

8.4 Doctor's Availability Acceptance Criteria

- **AC-07:** Doctors should configure their working schedules including setting start and end times or marking themselves as unavailable for appointments in the medical appointment system
- **AC-08:** When a doctor updates their schedule, this information should be visible in real-time on the receptionist's computer system hence avoiding clashes in the appointment book

8.5 Make Appointments Acceptance Criteria

- **AC-09:** When making appointments, the new application should prevent double-booking by avoiding scheduling patients at the same time or when the doctor has marked their schedule as unavailable
- **AC-10:** Users should be able to easily track or update patient status depending on whether they show up for their scheduled visits, cancel, or miss the appointments from the main dashboard

8.6 Prescriptions Acceptance Criteria

- **AC-11:** Doctors should electronically store patient prescriptions including drugs, dosages, and instructions for each medication in a secure database.
- **AC-12:** The medical information system should display a list of patient medical history including prescriptions and notes in chronological order
- **AC-13:** Users must be able to print soft copies of patient medical reports by exporting them into a PDF file format in the java swing desktop application

8.7 Performance, Usability, and Stability Acceptance Criteria

- **AC-14:** All database queries should display results, patient records, and appointment books within two seconds on the java swing interface
- **AC-15:** The desktop application should be stable enough to recover from unexpected errors, crashes, or connection losses during database access while saving the work automatically
- **AC-16:** All final checks must be performed by the stakeholders and hospital staff to confirm that all aspects of the new software meet the intended purpose and rules governing healthcare institutions

9. Project Plan

Whatever the nature of the software project is, it is essential to develop it appropriately. The Medical Appointment & Prescription proposal will be developed using the V-Model (Verification and Validation Model with Feedback Loops). This model allows testing at every stage of the development process with a possibility of going back and redesigning or re-coding some parts if needed.

9.1 Phase Wise Execution Strategy

The software development life cycle of the Medical Appointment & Prescription system will be executed in the following phases with a possibility of going back to the previous stage for correction or enhancement:

i. Phase 1: Requirement Analysis and Acceptance Testing Planning

- This stage will involve identifying the issues the manual maintenance of the appointment and prescription system causes to the stakeholders by interacting with them and researching similar systems online.
- This phase will conclude with the specification of objectives, requirements, and scope of the proposed project and the development of the Acceptance Test Plan. If some requirements have been missed, they can be added to the Requirements Specification by returning to this stage.

ii. Phase 2: System Design and System Testing Planning

- This phase will involve designing the Entity-Relationship model, Data Flow Diagrams, Use case diagrams, Class diagrams, Sequence diagrams, Activity diagrams, and MySQL schema of the proposed system.
- It will also involve developing the specification of the JavaSwing based Graphical User Interface required for the project and the System Test Plan. If any design flaws are identified at a later stage, this phase can be revisited to make the required corrections.

iii. Phase 3: Component Design and Integration Testing Planning

- The Component Design activities will involve capturing the logic of the different application components and DAOs (Database Access Objects) and record their interaction with external entities.
- This phase will also be used to develop the Integration Test Plan to guide the integration testing process so that if any issues are encountered while executing the application, this stage can be used to make the required alterations.

iv. Phase 4: Implementation (Coding) and Unit Testing

- This phase will involve implementing the proposed application using JavaSwing for the Graphical User Interface, Core Java for the application logic, and JDBC for connecting the application to the MySQL database.
- Each of the six modules – Login, Patient record management, Doctor record management, Appointment management, Prescription management, Medicine reminder, and Report generation will be coded and undergo the Unit Testing process. The coding and unit testing of the different modules will be carried out simultaneously before integration testing commences. If any flaws are detected during the unit testing process, the development team will make the required code revisions.

v. Phase 5: Validation, System Integration, and Delivery

- This phase will involve performing the integration and system tests as per the plans developed in earlier stages while compiling the modules into a complete desktop application that will help eliminate the problems caused by the manual system.
- By going back to the earlier stages, the development team will make the necessary revisions to address the issues detected during the validation process before preparing the final project report and user manual and delivering the project to the stakeholders.

9.2 Project Schedule (Gantt Chart)

The project begins with requirement analysis and project planning, followed by system and database design. The graphical user interface and application modules are then developed using Java Swing, Core Java, JDBC, MySQL, and NetBeans IDE. After implementation
Legend: ▉ = Activity performed during the corresponding week.

Activity	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8	W 9	W 10	W 11	W 12	W 13	W 14	W 15	W 16
Requirement Analysis	▉	▉														
Project Planning	▉	▉														
System Design (DFD, ER, UML)		▉	▉	▉												
Database Design			▉	▉												
User Interface Design				▉	▉											
Module Development					▉	▉	▉	▉	▉							
Database Integration							▉	▉								
Testing & Debugging									▉	▉	▉					
Report Generation										▉	▉	▉				
Documentation											▉	▉	▉	▉		
Final Testing													▉	▉		
Deployment & Presentation															▉	▉

9.3 Work Breakdown Structure (WBS)

The Work Breakdown Structure (WBS) illustrates the division of the Medical Appointment & Prescription System into smaller manageable activities for development and construction.

i Project initiation

- Requirement Gathering
- Project Planning

ii System Design

- Database Design
- UI Design
- Architecture Design

iii. Development

- User Login and Authentication
- Patient Management
- Doctor Management
- Appointment Scheduling
- Prescription Management
- Medicine Reminder
- Medical History
- PDF Report Generation

v. Testing & Validation (V-Model Verification Phase)

- Unit Testing
- Integration Testing
- Bug Fixing & Backtracking Refinemen

vi. Documentation

- SRS
- Design Documents & V-Model Traceability Matrix
- User Manual

vii. Deployment

- Final System Validation
- Project Submission
- Presentation

9.4 Resource Planning

- Proper resource planning helps in having the most efficient utilization of resources without putting off the time of project completion.
- It confirms the availability of hardware, software, development tools, and manpower at different stages of the project.
- The Medical Appointment & Prescription System has been developed using Java Swing, Core Java, JDBC, MySQL, and NetBeans IDE.
- Therefore, this project is cost-effective and efficient, considering using open-sourced software and hardware, making it an affordable desktop application that does not require advanced hardware and software.
- The team that initiated this project consists of 2 members, and each has their responsibilities in the development of this project.
- The faculty supervisor guides and supervises the work of the team members in line with project requirements.
- Team Member 1 is expected to work on the patient, doctor, user authentication, and database modules.
- On the other hand, Team Member 2 focuses on the appointment, prescription, medicine reminder, and report generation modules.

- Together they carry out other tasks such as database connectivity, system testing, bug fixing, improving the user interface, documentation, presentation, and deployment of the system.
- The development of the Medical Appointment & Prescription System takes place on ASUS Vivobook 15.
- Some requirements of this project include Windows 11, NetBeans IDE, JDK, MySQL Server, MySQL Workbench, JDBC, Git, GitHub, Microsoft Word, and Google Chrome.
- The development process of this project takes approximately five months, including requirement analysis, system design, database design, and interface building, followed by coding, testing, documentation, and final deployment.
- Finally, the financial expenditure on this project is minimal as most of the hardware and software required for this project are open-sourced, except for internet connectivity and printing costs involved.

9.5 Hardware Requirements

Medical Appointment & Prescription System requires a computer system that allows developing, testing, debugging, and documenting the application simultaneously without experiencing slow performance.
The project is being developed and tested on ASUS Vivobook 15 laptop.
Below are the required hardware specifications:

- Processor – 13th Gen Intel Core i5-13500H Processor (2.60 GHz)
- RAM – 16 GB DDR4 RAM (15.7 GB Usable)
- Storage – 512 GB NVMe SSD (477 GB Usable)
- Graphics – NVIDIA GeForce RTX 2050 (4 GB GDDR6) and Intel Iris Xe Graphics
- Display and Operating System – 15.6 Inch Full HD Display, Windows 11 Home Single Language (64-bit), Version 25H2
- Input and Connection – Keyboard, Touchpad, and Wi-Fi Broadband Internet Connection

9.6 Software Requirements

Medical Appointment & Prescription System requires a software platform that allows the application programming, testing, documentation, testing, debugging, and version control processes to take place without encountering compatibility issues.
The following are the software packages needed:

- Operating System – Windows 11 Home Single Language (64-bit)
- IDE – NetBeans IDE
- Programming Languages – Core Java For Application Programming
- Java Swing for Graphical User Interface (GUI)
- Development Kit – Java Development Kit (JDK 24 or higher)
- Database – MySQL Server for database management
- Database Administration and Modeling – MySQL Workbench
- Database Connectivity – JDBC Driver
- Version Control – Git and GitHub for version control and collaboration
- Documentation and Research – Microsoft Word for report writing and documentation and Google Chrome for documentation, download packages and research

9.7 Cost estimation

- No Software Purchase Cost – Java, NetBeans, MySQL, JDBC, and Git are open-sourced and free for academic use.
- No Infrastructure Cost – The project does not require cloud servers or workstations.
- No Labor Cost – The project is developed by a two-student team under the supervision of a faculty member.

- Low Operational Cost – Minimal operational expenditures are incurred as the development takes place on the available laptop, and the project team only spends a little on internet connectivity and printing.

9.8 Team Structure

Medical Appointment & Prescription System project will be developed by 2 students.
The 2 members will share responsibilities to ensure the project meets set standards and finishes on time.
Both will collaborate to deliver an efficient project according to the requirements.
The responsibilities of each member include:

Team Member 1: Authentication & Patient Module

This team member will design and develop the authentication and patient management systems.
This includes carrying out requirement analysis, designing the database, developing secure login and authentication, designing the registration forms for patients, and developing patient records, medical history systems, and connecting the developed modules to the database.
This team member will also perform system integration and documentation of the developed system.
Responsibilities:

- Requirement Analysis
- User Login and Authentication
- Patient Registration
- Patient Records Management
- Management of Medical History
- Database Design
- Database Connectivity
- Integration Support
- Documentation

Team Member 2: Doctor & Appointment Module

This team member will handle the design and develop doctor management and appointment systems.
The responsibilities include developing the doctor's registration system, managing the availability of doctors, developing the system for scheduling appointments, managing prescriptions, developing the medicine reminder system, tracking the status of appointments, generating reports in PDF format, testing, and documentation of the developed modules.

Responsibilities:

- Doctor Management
- Doctor's Availability
- Appointment Scheduling
- Prescription Management
- Prescription History
- Medicine Reminder
- Appointment Status
- PDF Report Generation
- Testing
- Documentation
- Common Responsibilities:
- Both team members will perform the below tasks:
- Database Integration
- Testing
- Bug Fixing

- Improving the User Interface
- Project Documentation
- Project Presentation
- Project Deployment

Effective communication and cooperation from both team members are needed to ensure that everyone delivers their responsibilities, leading to a successful project completion.

9.9 Development Schedule

Medical Appointment & Prescription System project is scheduled to take 4 ½ months to ensure each stage of the development process is adequately addressed.

i. August – Requirement Analysis and Planning

- The topic of the project will be finalized after going through the industry and identifying the problem that this project will solve.
- The objectives, scope, and functional requirements will be identified, and the Software Requirement Specification (SRS) document prepared.
- In addition, a literature survey will be carried out, and feasibility analysis conducted.

ii. September – System Design

- The second month will be dedicated to designing the system architecture, developing an ER Diagram, DFDs, Use Case Diagram, Class Diagram, Sequence Diagrams, Activity Diagram, and system architecture.
- The UI design will also be developed before building the application.

iii. October – Application Development

- The third month will involve coding the application using Java Swing and Core Java as the programming languages, JDBC as the database driver, and MySQL as the database.
- The following modules will be developed:
- User Login and Authentication
- Patient Management
- Doctor Management
- Appointment Scheduling
- Prescription Management
- Medical History
- Medicine Reminder

iv. November – Integration and Testing

- The developed modules will be integrated into one complete application.
- Various tests, including functional and integration testing, will be carried out, and the bugs fixed.
- Additionally, the report generation in PDF format and other remaining minor modules will also be developed in this phase.

v. December – Documentation and Final Submission

- The last month will focus on preparing the project documentation, presentation slides, and demo video for submission.
- Testing will be carried out to ensure every module functions correctly before the final submis